

Assignment 3: Model-Based Reinforcement Learning

Reinforcement Learning — Leiden University
Windy Gridworld Environment | Dyna & Prioritized Sweeping

1. Dyna

1.1 Methodology

Dyna is a model-based reinforcement learning algorithm that combines real environment interaction with simulated planning. At each timestep, the agent selects an action using an epsilon-greedy policy and observes the next state and reward. This real experience is used in two ways: first, to update a learned model of the environment; and second, to perform a standard Q-learning update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

The environment model is built by tracking two tables: the transition count $n(s,a,s')$, which counts how many times the agent moved from state s to s' via action a , and the reward sum $R_sum(s,a,s')$, which accumulates the total reward observed for each (s,a,s') triple. From these tables, the agent estimates the transition probability and expected reward as:

$$p^{\wedge}(s'|s,a) = n(s,a,s') / \sum n(s,a,s') \quad r^{\wedge}(s,a,s') = R_sum(s,a,s') / n(s,a,s')$$

After the real step and Q-update, the agent performs K planning steps. In each planning step, a previously visited (s,a) pair is chosen at random, a next state s' is sampled from the estimated transition model, and another Q-learning update is applied using the simulated experience. This planning allows the agent to learn much faster than standard Q-learning, since each real step is effectively multiplied into $K+1$ updates.

1.2 Results

Figure 1 shows the learning curves for Dyna with different planning budgets ($K = 1, 3, 5$) compared to a Q-learning baseline ($K = 0$), in both the stochastic environment ($wind = 0.9$) and the deterministic environment ($wind = 1.0$). Each curve is averaged over 20 independent repetitions of 10,001 timesteps.

[Figure 1: *dyna_stochastic.png* — Dyna in stochastic environment]

[Figure 2: *dyna_deterministic.png* — Dyna in deterministic environment]

1.3 Interpretation

In both environments, increasing the number of planning updates leads to faster learning. With $K = 5$, the agent reaches a high episode return significantly earlier than with $K = 1$ or the Q-learning baseline. This is expected: each real step generates K additional simulated updates, so the effective sample efficiency improves with K .

In the deterministic environment, the effect is particularly pronounced. All Dyna variants converge to a similar final performance, but with $K = 5$ this happens nearly four times faster than Q-learning. The deterministic environment is ideal for Dyna because the model is accurate — the wind always blows, so every simulated transition is reliable.

In the stochastic environment, the gains from planning are still clear but slightly smaller. Occasionally, the model produces inaccurate simulated transitions (since the wind only blows 90% of the time), which adds some noise to the planning updates. Nevertheless, Dyna with $K = 5$ still learns substantially faster than Q-learning.

2. Prioritized Sweeping

2.1 Methodology

Prioritized Sweeping (PS) builds on the same model as Dyna but replaces the random planning step selection with a priority queue. The key idea is that not all (s,a) pairs are equally worth updating — we should focus on those where the current Q-value estimate is most inaccurate.

After each real environment step, the agent computes a priority p for the current (s,a) pair, defined as the absolute temporal difference (TD) error:

$$p = | r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) |$$

If this priority exceeds a small threshold $\theta = 0.01$, the pair (s,a) is inserted into a priority queue.

During planning, the agent repeatedly pops the highest-priority (s,a) from the queue, performs a Q-update using the model, and then checks all predecessor state-action pairs $(\tilde{s}, \tilde{a})$ that can lead to s . For each such predecessor, a new priority is computed and if it exceeds θ , it is added to the queue. This backward propagation of value information is what makes PS efficient.

2.2 Results

[Figure 3: *ps_stochastic.png* — Prioritized Sweeping in stochastic environment]

[Figure 4: *ps_deterministic.png* — Prioritized Sweeping in deterministic environment]

2.3 Interpretation

In the deterministic environment, Prioritized Sweeping shows impressive performance. With $K = 5$, PS converges even faster than Dyna with the same planning budget. This is because the environment is perfectly predictable, so the priority estimates are accurate and the backward sweep efficiently propagates reward information from the goal state backward through the state space.

In the stochastic environment, however, PS performs noticeably worse than expected. The Q-learning baseline eventually surpasses PS with higher planning budgets in terms of final performance. This can be explained by the noise introduced by stochasticity: the priority estimates become less reliable when transitions are random, and the backward sweep can propagate incorrect information. PS is most powerful when the model is accurate, which requires a deterministic or near-deterministic environment.

3. Comparison

3.1 Learning Curves

Figures 5 and 6 compare the best-performing Dyna ($K = 5$), the best-performing PS ($K = 5$), and the Q-learning baseline in both environments.

[Figure 5: comparison_deterministic.png — Comparison in deterministic environment]

[Figure 6: comparison_stochastic.png — Comparison in stochastic environment]

In the deterministic environment, PS clearly outperforms Dyna, learning the fastest of all three methods. Both model-based approaches comfortably beat the Q-learning baseline. In the stochastic environment, Dyna outperforms PS, and interestingly, Q-learning eventually catches up with — and surpasses — PS by the end of training. This highlights how sensitive PS is to environment stochasticity.

3.2 Runtime Comparison

The table below shows the average runtime per repetition (10,001 timesteps, 5 repetitions used for timing):

Algorithm	Stochastic (s/rep)	Deterministic (s/rep)
Q-learning	3.92	4.13
Dyna ($K=5$)	6.59	4.34
Prioritized Sweeping ($K=5$)	12.46	11.31

Q-learning is the fastest algorithm since it performs only one update per timestep. Dyna is moderately slower because of the additional $K = 5$ planning updates. PS is the slowest by a significant margin: in addition to planning updates, it must scan all state-action pairs to find predecessors for the backward sweep, which is computationally expensive in larger state spaces.

Interestingly, the runtime of PS does not strongly depend on whether the environment is stochastic or deterministic ($\approx 12s$ vs $\approx 11s$), while Dyna is somewhat slower in the stochastic case. This is because in the stochastic case, the agent tends to explore more, visiting more (s,a) pairs during planning.

4. Reflection

4.1 Model-Based vs. Model-Free RL

A key strength of model-based RL is sample efficiency. By building and reusing a model of the environment, the agent can extract far more learning signal from each real interaction. This is particularly valuable when real interactions are expensive — for example, in robotics or clinical settings. In our experiment, Dyna with $K = 5$ reached the same performance as Q-learning in roughly one-fifth of the timesteps.

However, model-based methods also have a clear weakness: they depend on the accuracy of the learned model. In stochastic environments, the model is only an approximation, and planning with an inaccurate model can lead to suboptimal or even harmful updates. In complex real-world environments with continuous state spaces, building an accurate model becomes even harder. Model-free methods like Q-learning are more robust in these settings, since they learn directly from experience without this intermediary.

4.2 Dyna vs. Prioritized Sweeping

Both algorithms use the same underlying model, but differ in how they select states for planning. Dyna's random selection is simple and robust, working well even in stochastic environments. PS's priority-based selection is smarter and more efficient when the model is accurate, as seen in the deterministic results.

In terms of which performed better overall: PS wins in the deterministic setting, while Dyna wins in the stochastic setting. If the environment is known to be close to deterministic, PS is the better choice. For noisy environments, Dyna is more reliable. In principle, one could combine both ideas — using priorities to focus planning, but with added robustness to model errors — for example by occasionally mixing in random planning steps alongside priority-based ones.

4.3 Exploration and Q-Value Initialization

All $Q(s,a)$ values were initialized to 0, while every non-goal step yields a reward of -1. This means the agent initially has no preference between actions, since all Q-values are equal. However, as soon as the agent reaches the goal for the first time and receives +100, those Q-values become very positive, making exploration of new paths less likely. In effect, initializing Q-values to 0 with a negative step reward is a form of pessimistic initialization — it slightly encourages the agent to seek the goal quickly, since staying in the grid is always penalized.

This is quite different from optimistic initialization (e.g., initializing all Q-values to a large positive number), which would strongly encourage exploration by making the agent expect high rewards everywhere until it learns otherwise. With optimistic initialization, the agent would systematically visit every state before settling on an optimal policy — a very different exploration dynamic.

Setting the default reward to 0 would remove this penalty signal. In that case, the agent would have no intrinsic motivation to reach the goal quickly, and early exploration would be much more random. This could slow down learning considerably, since the agent would not strongly prefer actions that seem to lead toward the goal. The -1 per step reward acts as a simple but effective way to encourage efficiency in reaching the goal.